SPARQL Property Paths

Simon Frost

Overview

SPARQL property paths let you navigate RDF graphs by following chains of predicates without writing multiple triple patterns. RDFLib.jl supports the full SPARQL 1.1 property path syntax.

```
using RDFLib

# Build an example graph: a small social network and class hierarchy
g = RDFGraph()
ex = Namespace("http://example.org/")

# People and friendships (a chain)
add!(g, Triple(ex("alice"), ex("knows"), ex("bob")))
add!(g, Triple(ex("bob"), ex("knows"), ex("carol")))
add!(g, Triple(ex("carol"), ex("knows"), ex("dave")))
add!(g, Triple(ex("dave"), ex("knows"), ex("eve")))

# Type hierarchy
add!(g, Triple(ex("Student"), RDFS.subClassOf, ex("Person")))
add!(g, Triple(ex("GradStudent"), RDFS.subClassOf, ex("Student")))
add!(g, Triple(ex("alice"), RDF.type, ex("GradStudent")))

# Additional properties
add!(g, Triple(ex("alice"), ex("email"), Literal("alice@example.org")))
add!(g, Triple(ex("alice"), FOAF("name"), Literal("Alice")))
add!(g, Triple(ex("bob"), FOAF("name"), Literal("Bob")))
add!(g, Triple(ex("carol"), FOAF("name"), Literal("Carol")))
add!(g, Triple(ex("dave"), FOAF("name"), Literal("Dave")))
add!(g, Triple(ex("eve"), FOAF("name"), Literal("Eve")))
println("Graph has $(length(g)) triples")
```

```
Graph has 13 triples
```

Sequence Path (/)

The / operator chains predicates. `ex:knows/ex:knows` matches two hops:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:alice ex:knows/ex:knows ?person .
    }
""")
println("Alice knows-of-knows:")
for row in results
    println("  $(row["person"])")
end
```

```
Alice knows-of-knows:
  URIRef("http://example.org/carol")
```

Three-hop chain:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:alice ex:knows/ex:knows/ex:knows ?person .
    }
""")
println("Three hops from Alice:")
for row in results
    println("  $(row["person"])")
end
```

```
Three hops from Alice:
  URIRef("http://example.org/dave")
```

Alternative Path (|)

The | operator matches either predicate:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?s ?info WHERE {
        ?s ex:email|foaf:name ?info .
    }
""")
println("Email or name:")
for row in results
    println("  $(row["s"]) → $(row["info"])")
end
```

```
Email or name:
  URIRef("http://example.org/alice") → Literal("alice@example.org")
  URIRef("http://example.org/alice") → Literal("Alice")
  URIRef("http://example.org/bob") → Literal("Bob")
  URIRef("http://example.org/carol") → Literal("Carol")
  URIRef("http://example.org/dave") → Literal("Dave")
  URIRef("http://example.org/eve") → Literal("Eve")
```

Transitive Closure (+ and ∗)

+ matches one or more hops (transitive closure):

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:alice ex:knows+ ?person .
    }
""")
println("All people Alice can reach (1+ hops):")
for row in results
    println("  $(row["person"])")
end
```

```
All people Alice can reach (1+ hops):
  URIRef("http://example.org/bob")
  URIRef("http://example.org/carol")
  URIRef("http://example.org/dave")
  URIRef("http://example.org/eve")
```

`*` matches zero or more hops (includes the start node):

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:alice ex:knows* ?person .
    }
""")
println("All people reachable from Alice (0+ hops, includes self):")
for row in results
    println("  $(row["person"])")
end
```

```
All people reachable from Alice (0+ hops, includes self):
  URIRef("http://example.org/alice")
  URIRef("http://example.org/bob")
  URIRef("http://example.org/carol")
  URIRef("http://example.org/dave")
  URIRef("http://example.org/eve")
```

Optional Path (`?`)

`?` matches zero or one hops:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:alice ex:knows? ?person .
    }
""")
println("Alice and direct friends (0 or 1 hop):")
for row in results
    println("  $(row["person"])")
end
```

```
Alice and direct friends (0 or 1 hop):
  URIRef("http://example.org/bob")
  URIRef("http://example.org/alice")
```

Inverse Path (^)

^ reverses the direction — find who knows someone:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:carol ^ex:knows ?person .
    }
""")
println("Who knows Carol (inverse):")
for row in results
    println("  $(row["person"])")
end
```

```
Who knows Carol (inverse):
  URIRef("http://example.org/bob")
```

Combine inverse with transitive to find all people upstream:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?person WHERE {
        ex:eve ^ex:knows+ ?person .
    }
""")
println("Who can reach Eve (inverse transitive):")
for row in results
    println("  $(row["person"])")
end
```

```
Who can reach Eve (inverse transitive):
  URIRef("http://example.org/dave")
  URIRef("http://example.org/carol")
  URIRef("http://example.org/bob")
  URIRef("http://example.org/alice")
```

Class Hierarchy Traversal

Property paths are particularly useful for navigating class hierarchies:

```
results = sparql_query(g, """
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    PREFIX ex: <http://example.org/>
    SELECT ?class WHERE {
        ex:GradStudent rdfs:subClassOf* ?class .
    }
""")
println("GradStudent is a (transitive):")
for row in results
    println("  $(row["class"])")
end
```

```
GradStudent is a (transitive):
  URIRef("http://example.org/GradStudent")
  URIRef("http://example.org/Student")
  URIRef("http://example.org/Person")
```

Find all instances of a class or its subclasses:

```
results = sparql_query(g, """
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    PREFIX ex: <http://example.org/>
    SELECT ?instance WHERE {
        ?instance rdf:type/rdfs:subClassOf* ex:Person .
    }
""")
println("All instances of Person (including subclasses):")
for row in results
    println("  $(row["instance"])")
end
```

```
All instances of Person (including subclasses):
  URIRef("http://example.org/alice")
```

Negated Property Set (`!`)

`!` matches any predicate except the listed ones:

```
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?s ?p ?o WHERE {
        ex:alice !(rdf:type|ex:knows) ?o .
    }
""")
println("Alice's properties (excluding rdf:type and ex:knows):")
for row in results
    println("  $(row["o"])")
end
```

```
Alice's properties (excluding rdf:type and ex:knows):
  Literal("alice@example.org")
  Literal("Alice")
```

Combining Operators

Property path operators can be combined freely:

```
# Find names of everyone reachable from Alice
results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?name WHERE {
        ex:alice ex:knows+ / foaf:name ?name .
    }
""")
println("Names of people Alice can reach:")
for row in results
    println("  $(row["name"])")
end
```

```
Names of people Alice can reach:
  Literal("Bob")
  Literal("Carol")
  Literal("Dave")
  Literal("Eve")
```

Summary of Path Operators

Operator	Syntax	Description
Sequence	`p/q`	Follow p then q
Alternative	`p|q`	Follow p or q
One or more	`p+`	Transitive closure (1+ hops)
Zero or more	`p*`	Reflexive transitive closure
Zero or one	`p?`	Optional step
Inverse	`^p`	Reverse direction
Negated set	`!(p|q)`	Any predicate except p and q